Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin tarihi 7 Ekim 2020 Çarşamba. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:06'da başlar. İlk dakikalar katılımcı beklemesidir.
  - 00:49 ile 00:58 arasında ve 01:35 ile 01:41 arasında ara var.
  - Hoca tahtayı not uygulamasında (tablet) kullanıyor, slaytlar kitabın slaytları.
- Dersin işleyişi ve değerlendirme
  - Ders iki grupta veriliyor. Grup 1'i Elif Hoca, Grup 2'yi (bu kayıt) dersin hocası veriyor. İki grup ödev, quiz ve sınavlarda senkron ilerliyor.
  - Ana kitap: Levitin, *Introduction to the Design and Analysis of Algorithms*. Destek kitapları: Skiena ve Sedgewick'in kitapları.
  - Plan: bir vize (büyük ihtimalle 9. hafta), 7-8 kısa quiz, 3-4 ödev ve bir dönem projesi. Yıl içi %60, final %40.
  - Ödevler veri yapıları dersindeki gibi kod ve tasarım ağırlıklı. Uygulama dili şimdilik C.
- 14 haftalık konu planı
  - Algoritma analizinin temelleri, verimlilik parametreleri.
  - Asimptotik analiz (asymptotic analysis).
  - İteratif ve özyinelemeli (recursive) algoritmaların analizi.
  - Böl ve yönet (divide and conquer), azalt ve yönet (decrease and conquer), dönüştür ve yönet (transform and conquer).
  - Graf algoritmaları, geri izleme (backtracking), dal ve sınır (branch and bound).
  - Dinamik programlama (dynamic programming), hash.
  - Zaman-yer ödünleşimi (time-space trade-off), P, NP, NP-complete ve NP-hard.
- Algoritma kavramı
  - Algoritma: bir problemi çözmek için belirsiz olmayan (unambiguous) adımlar dizisi. Sonlu sürede, geçerli girdi için doğru çıktı üretmelidir.
  - Aynı problem için farklı algoritmalar olabilir. Örnek: en büyük ortak bölen (EBOB, gcd). Çarpanlara ayırma yöntemi ve Öklid yöntemi. $\gcd(60, 24) = 12$.
- Algoritma karmaşıklığını etkileyen problem parametreleri
  - Eleman sayısı (input size, n).
  - Elemanların içeriği ve geliş sırası. Bu yüzden en iyi (best), ortalama (average) ve en kötü (worst) durum ayrı incelenir.
- Algoritmayı ifade etme yolları
  - Akış diyagramı (flowchart), sözlü anlatım, sözde kod (pseudocode), programlama dili.
  - Bu derste ağırlıklı olarak sözde kod kullanılacak.
- Algoritma çalışmasının temel adımları
  - Problemi anlamak, yaklaşımı (tam veya yaklaşık çözüm) ve tasarım tekniğini seçmek, algoritmayı tasarlamak.
  - Doğruluğu göstermek, verimliliği (zaman ve yer) analiz etmek, kodlamak.
- Algoritma tasarım stratejileri: kaba kuvvet (brute force), böl ve yönet, azalt ve yönet, dönüştür ve yönet, açgözlü (greedy), dinamik programlama, geri izleme, dal ve sınır, zaman-yer ödünleşimi.
- Algoritmayı değerlendirme ölçütleri
  - Basitlik (simplicity), genellik (generality), zaman verimliliği (time efficiency), yer verimliliği (space efficiency).
  - En önemli ölçüt doğruluktur. Hız ve bellek ikinci sıradadır.

  - Optimallik: Problemin alt sınırına (lower bound) ulaşan bir algoritma varsa daha iyisini aramaya gerek yoktur.
- Durum senaryoları ve örnekler (tahtadan)
  - Sıralı arama (sequential/linear search), sırasız dizi: en iyi durum $O(1)$, en kötü durum $O(n)$.
  - Sıralı arama, sıralı dizi: en kötü durum yine $O(n)$, en iyi durum $O(1)$.
  - En büyük elemanı bulma, sırasız dizi: her durumda bütün elemanlara bakılır, $O(n)$.
  - En büyük elemanı bulma, sıralı dizi: her durumda $O(1)$. Sıralama yönü biliniyorsa ilk veya son eleman alınır.
- Deneysel (empirical) ve kuramsal (theoretical) analiz
  - Deneysel analiz: algoritmayı kodla, farklı girdi boyutlarında süre ölç, grafiğini çiz.
  - Kuramsal analiz: sözde kod üzerinden temel işlemleri (primitive operations) say, n cinsinden fonksiyon yaz.
- Sözde kod kuralları ve temel işlemler
  - Atama için $\leftarrow$, eşitlik için $=$. Blok yapısı parantezle değil girintiyle (indentation) gösterilir.
  - Temel işlemler: ifade değerlendirme (aritmetik ve karşılaştırma), atama, diziye erişim, metot çağırma, metottan dönme.
- `arrayMax` örneği: işlem sayısı $7n - 1$, yani doğrusal (linear).

Hocanın Özellikle Vurguladığı Kısımlar

- Not tutun.
  - Hoca slayt ve tahtadaki bilgileri defterinize geçirmenizi istiyor. Defterler zaman zaman istenip küçük bir puanla ödüllendirilebilir.
  - Kayıtlar ve slaytlar paylaşılsa da kendi defterinizi tutmanızı öneriyor.
- Ödev ve projeye sahip çıkın.
  - Geçen dönem veri yapılarında yaklaşık 80 öğrenciden 20'si hiç ödev yapmadan dönemi bitirdi.
  - Birbirinizden bilgi almak serbest. Kopya tespit edilirse ödev sıfır sayılır.
- Derse zamanında ve hazırlıklı gelin.
  - Ders saat 13:00'te başlar. Haftalık konuya önceden göz atın.
  - Kameranızı gerektiğinde açabilecek durumda olun. Mikrofonla sesli katılım beklenir.
- Konuşulan karmaşıklığın hangi durum için olduğunu mutlaka söyleyin.
  - "Sıralı dizide en büyük eleman $O(1)$" derken "bütün durumlarda" demezseniz cevap yanlış sayılır.
  - Sıralı aramayı sorduğunda "$O(n)$" cevabı tek başına yanlıştır. Önce senaryo belirtilmelidir.
- Bu derste en çok en kötü durum (worst case) incelenecek.
  - En iyi durum tek başına işe yaramaz. Ortalama durumun hesabı genelde zordur.
  - En kötü durum, sistem tasarımında esas alınan durumdur. Örnek: ders kayıt sistemi, aynı anda giriş yapan en kalabalık sınıfa göre tasarlanır. ÖSYM sonuçlarında aynı anda giren 2 milyon kişi düşünülür.
- Sözde kod yazmayı şimdiden öğrenin. Sınav ve quizlerde sözde kodla karşılaşacaksınız.
- Bir algoritmanın büyüme derecesini döngüler belirler.
  - Tek döngü varsa doğrusal, iç içe iki döngü varsa genelde n^2 olur.
- Karşılaştırmaya dayalı sıralama algoritmaları $n \log n$'den daha iyi olamaz. Merge sort ve heap sort bu yüzden en kötü durumda optimaldir.

Kısa Tekrar Notları

- Algoritma: belirsiz olmayan, sonlu sürede biten, doğru çıktı üreten adımlar dizisi.
- Karmaşıklığı etkileyen problem parametreleri: girdi boyutu n ve girdinin içeriği ile sırası.
- Üç senaryo: en iyi, ortalama, en kötü durum. Ders boyunca en kötü durum esas alınır.
- Dört gösterim: akış diyagramı, sözlü anlatım, sözde kod, programlama dili.
- Değerlendirme ölçütleri: doğruluk, basitlik, genellik, zaman verimliliği, yer verimliliği, optimallik.
- Deneysel analiz: kodlama gerekir, aynı donanım ve yazılım ortamı gerekir, bütün girdileri deneyemezsiniz.

- Kuramsal analiz: sözde kod üzerinden, donanımdan bağımsız, bütün girdileri kapsar.
- Temel işlemler: aritmetik ve karşılaştırma, atama, dizi erişimi, metot çağırma ve dönme.
- `arrayMax`: $T(n) = 7n - 1$, doğrusal büyüme.

Detaylı Açıklamalar

Algoritma analizinde amaç, iki algoritmayı donanımdan bağımsız biçimde karşılaştırmaktır. Bunun için algoritma, girdi boyutu n'nin bir fonksiyonu olarak ifade edilir. Hoca bunu ileride büyüme derecesi (order of growth) ve asimptotik gösterimlerle bağlayacağını söyledi.

Arama ve en büyük eleman örnekleri

Hoca tahtaya iki dizi yazdı: sırasız `1 5 3 2 4 8` ve sıralı `1 2 3 4 5 8`.

Problem	Dizi	En iyi durum	En kötü durum
Sıralı arama	Sırasız	$O(1)$	$O(n)$
Sıralı arama	Sıralı	$O(1)$	$O(n)$
En büyük eleman	Sırasız	$O(n)$	$O(n)$
En büyük eleman	Sıralı	$O(1)$	$O(1)$

- Aramada aranan eleman ilk sırada olabilir. Bu yüzden en iyi durum $O(1)$'dir.
- En büyük elemanı sırasız dizide bulmak için bütün elemanlara bakmak gerekir. Bu yüzden en iyi durum da $O(n)$'dir.
- Dizi sıralıysa ve sıralama yönü biliniyorsa en büyük eleman ilk ya da son elemandır.
- İkili arama (binary search) kullanılsaydı sıralı dizide en kötü durum $O(\log n)$ olurdu. Ama soru sıralı (linear) arama içindi.

Deneysel analiz

İki algoritmayı aynı girdilerle çalıştırıp süreleri ölçebilirsiniz. Dikkat edilecekler:

- Girdiler aynı olmalı.
- İki algoritma aynı donanım ve yazılım ortamında çalışmalı.
- Algoritmaları önce kodlamanız gerekir.
- Bütün olası girdileri deneyemezsiniz. Sonuç yalnız denenen girdiler için geçerlidir.

Kuramsal analiz ve temel işlem sayma

Slayttaki `arrayMax` sözde kodu ve işlem sayıları:

```
Algorithm arrayMax(A, n)                # işlem
  currentMax <- A[0]                    2
  for i <- 1 to n - 1 do                2 + n
    if A[i] > currentMax then           2(n - 1)
      currentMax <- A[i]                2(n - 1)
    { increment counter i }             2(n - 1)
  return currentMax                     1
                                Toplam: 7n - 1
```

- Toplam: $2 + (2 + n) + 3 \cdot 2(n - 1) + 1 = 7n - 1$. Bu, en kötü durumda (her adımda `currentMax` güncellenirse) işlem sayısıdır.
- En hızlı temel işlemin süresi a, en yavaşınınki b ise çalışma süresi $a(7n - 1) \leq T(n) \leq b(7n - 1)$ arasındadır.
- Donanım değişirse a ve b değişir, ama büyüme derecesi doğrusal kalır.

Ders sonrası notlar

- **Polinom zaman ve sonlu süre:** Hoca, algoritmanın "istenilen sürede bitmesi"ne polinom zaman (polynomial time) dedi. Algoritma tanımındaki şart yalnızca sonlu sayıda

adımda bitmektir. Polinom zaman ayrı bir kavramdır ve verimlilik sınıfını anlatır. Üstel zamanlı bir algoritma da algoritmadır.

- **Sıralamanın alt sınırı:** "Karşılaştırmalı sıralama $n \log n$'den daha düşük olamaz" ifadesi en kötü durum için doğrudur: alt sınır $\Omega(n \log n)$'dir. En iyi durumda insertion sort gibi algoritmalar $O(n)$ olabilir.
- **Selection sort:** Derste selection sort'un optimal bir çözüm olduğu söylendi (01:31:34). Bunu büyük olasılıkla hoca, sınıfa soru olarak ortaya attı. Selection sort her durumda $\Theta(n^2)$ karşılaştırma yapar. Bu yüzden optimal değildir; merge sort ve heap sort en kötü durumda $\Theta(n \log n)$ ile optimaldir.
- **`arrayMax` işlem sayısı:** Slayttaki toplam $7n - 1$ doğrudur. Sayım, karşılaştırmanın her seferinde doğru çıktığı en kötü durum içindir. En iyi durumda (ilk eleman en büyükse) atama satırı hiç çalışmaz ve toplam $5n + 1$ olur.
- **Kitap adları:** Altyazıdaki "Stephen Scanlon" Steven Skiena'dır (*The Algorithm Design Manual*). "Robert Sedgwick" Robert Sedgewick'tir (*Algorithms*).
- **Altyazı hataları:** "Pissoda code", "püsöde kod" sözde kod (pseudocode); "Divide-Concure" divide and conquer; "breach and bounce" branch and bound; "8-Quint" 8 vezir (8 queens); "nebsek" knapsack (sırt çantası) problemidir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.